Design Optimization Strategy

To understand our optimization strategy, we must first explain the origin of our metric. We evaluate our design across 60 test loads. These scenarios span the full operating range of the robot, specifically varying the separation distance between the two applied forces. This physical context is crucial because the difficulty of the sensing task changes with distance: when forces are applied very close together (small separation), they become mechanically indistinguishable, while forces far apart (large separation) are easy to tell apart.

For each of these 60 test loads, we compute a Jacobian matrix, a mathematical map relating the applied forces to the raw sensor measurements. This 6x6 matrix can be thought of as determining the sensitivity of our system and amplifies the physical input (test load) $F = [F_1, s_1, \theta_1, F_2, s_2, \theta_2]$ into a sensor reading $W = [T_x, T_y, T_z, F_x, F_y, F_z]$. To quantify this sensitivity, we calculate the minimum singular value $\boldsymbol{\sigma_{min}}$, the system's weakest direction, where it is most "numb." A low value means some combination of forces could go nearly undetected, even if large. Maximizing this worst-case sensitivity is formally known as E-Optimality.

However, a raw singular value is just an abstract number. To give it physical meaning, we scale the Jacobian by the "noise" of our sensors. In our context, this noise is not just electronic static, it is a composite value derived from the three dominant sources of uncertainty in our hardware: static sensor noise (the electronic floor), mechanical hysteresis (the memory in the sensor's return to zero), and angular application error (the variation caused by forces not being applied perfectly perpendicular). We selected these three because they represent the fundamental limits of precision that cannot be simply calibrated away. Because these error sources are independent and fluctuate unpredictably, we combine them using the Root Sum Square (RSS) method, squaring each component, summing them, and taking the square root, to estimate a total "Random Noise Floor."

$$\sigma_{total} = \sqrt{\sigma_{static}^2 + \sigma_{hysteresis}^2 + \sigma_{angle}^2}$$

This SNR calculation focuses only on random noise (precision) rather than systematic error (accuracy). We do this because systematic errors, like a sensor that always reads 1N too high, can theoretically be calibrated out in software. Random noise, however, represents the fundamental "blur" or uncertainty limit of the hardware that cannot be removed. By treating hysteresis and angle variation as random noise, we ensure the optimization targets a design that is robust even when the system behaves unpredictably. This normalization converts the singular value into a unitless Signal-to-Noise Ratio (SNR). A value of $\boldsymbol{\sigma_{min}} < \mathbf{1}$

means the signal is smaller than this random noise floor (unobservable), while $\boldsymbol{\sigma_{min}} > \mathbf{1}$ means the signal is distinguishable. Therefore, the threshold of $\boldsymbol{\sigma_{min}} \approx \mathbf{1}$ represents the "edge of working," marking the critical transition where the robot's measurements finally become reliable.

This leaves us with 60 separate "grades" (one SNR for each separation distance), but the optimizer needs a single final score to decide if a design is good or bad. We cannot simply add these numbers up because value depends on context, and more is not always better. For example:

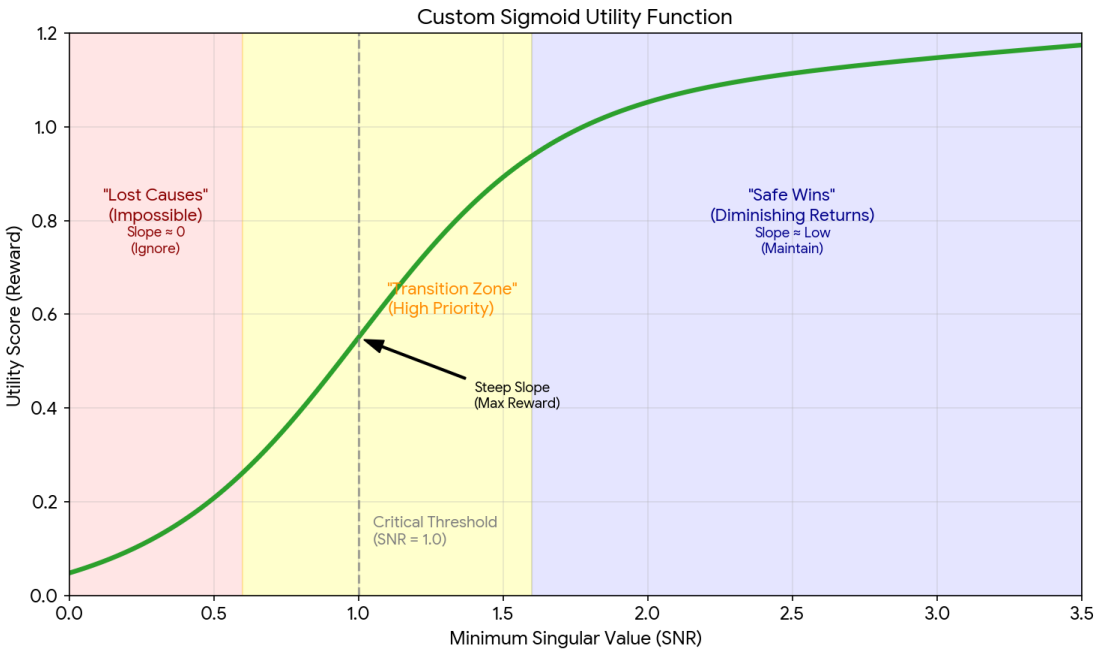
- Improving an "impossible" case from an SNR of 0.001 to 0.002 is mathematically double, but physically useless. The robot is still blind.
- Improving a "safe" case from an SNR of 10 to 20 is numerically huge, but practically unnecessary. The robot was already working perfectly.
- The only truly valuable improvement is pushing a case from 0.9 (unreliable) to 1.1 (reliable).

To capture this nuance, we need a mathematical rule (an activation function) to translate these raw SNRs into a meaningful "utility" score. Think of this function as the "grading rubric" we give the optimizer.

We chose a custom Sigmoid activation function because standard "textbook" grading rubrics fail to handle our physical constraints.

First, we rejected the simple "Linear" rubric, which maximizes the average performance. Averages are dangerous because they allow "compensation": the optimizer could leave the robot completely blind at small separations (a failure) but hide that fact by making the large separation cases super-sensitive. A robot that works perfectly 50% of the time and fails 50% of the time might have a good "average," but it is a bad design.

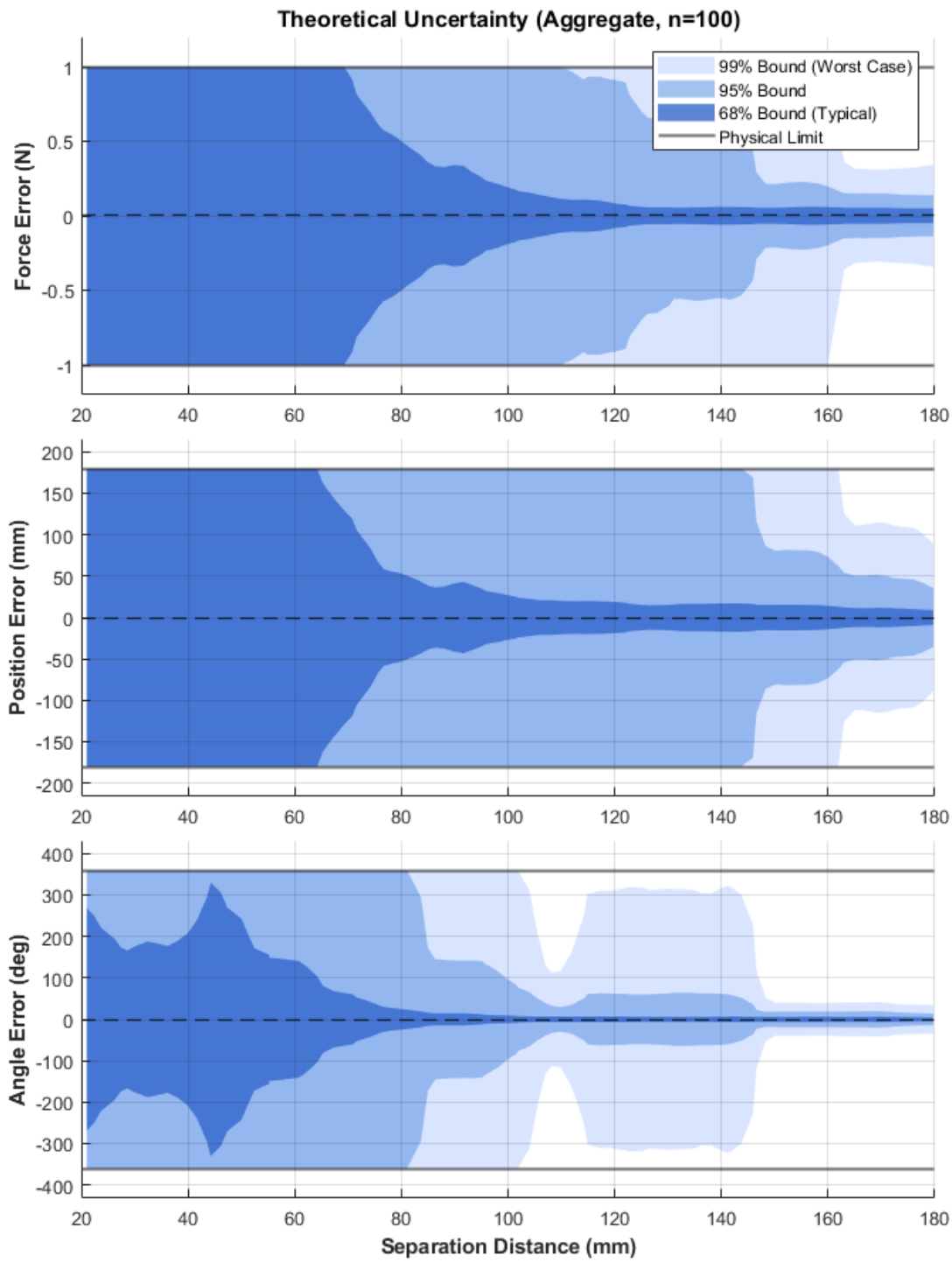
Second, we rejected strict rubrics like fixing the worst-case (or Log-Sum). These follow a "weakest link" philosophy: they force the optimizer to focus entirely on the absolute lowest score. In our system, the lowest scores naturally occur at very small force separations where physics makes high accuracy impossible. If we used these methods, the optimizer would waste 100% of its effort trying to squeeze a tiny improvement out of these "dead zones," while ignoring the rest of the workspace.



By contrast, our Sigmoid function acts as a strategic filter. Physically, it is an "S-shaped" curve centered exactly at our critical SNR threshold of 1.0. The curve is steep in the middle (roughly between SNR 0.5 and 2.0) but flattens out at the top and bottom. Because the curve is flat at the bottom, it ignores the "lost causes" (impossible small separations). Because it is flat at the top, it ignores the "safe wins" (easy large separations). Instead, the steep middle section forces the optimizer to spend its budget exclusively on the "transition zone", those cases right on the edge of becoming observable. This ensures the design maximizes the total number of usable configurations, rather than futilely fighting physics to fix a dead zone.

```
%% OBJECTIVE FUNCTION
function cost = objective_scalar(x, test_forces, design)
    % Update Design
    S = update_design(x, design, false);

    % Split inputs
    force_inputs = test_forces(:, 1:6);
    tau_arrays = test_forces(:, end-2:end);

    % Get force_to_wrench jacobians for all test loads
    J_cell = compute_jacobian(force_inputs, tau_arrays, S);

    % Sigmoid parameters
    target_threshold = 3.0;  % Target: The specific singular value where we want the transition to happen.
    k = 2.0;  % Steepness (k): Controls width of the "Transition Zone." Higher k = stricter, binary "Pass/Fail". Lower
    leak_slope = 0.01; % Leak (alpha): The gentle slope that keeps the optimizer moving even when the Sigmoid is flat.

    % Initialize total score
    total_score = 0;

    % Loop through test load jacobians
    for i = 1:size(J_cell,1)
        s_vals = svd(J_cell{i}); % Get singular values
        min_sv = s_vals(end); % Get min singular value

        % 1. Calculate margin
        margin = min_sv - target_threshold;

        % 2. Sigmoid score (The "Strategic Filter") S(x) = 1 / (1 + e^-k(x))
        % This naturally saturates to 1 (Safe Win) and 0 (Lost Cause)
        sigmoid_part = 1 / (1 + exp(-k * margin));

        % 3. Leak score (The "Guidance")
        % Adds a tiny gradient everywhere so the optimizer is never truly blind.
        leak_part = leak_slope * min_sv;

        % Total utility
        total_score = total_score + (sigmoid_part + leak_part);
    end

    % Negate for Minimizer (Maximize Score = Minimize Cost)
    cost = -total_score;
end
```

Once optimized, we must verify the design's performance in real-world units (Newtons, mm, degrees). We do this by calculating the "Typical Error" regions (shown in dark blue), derived directly from the covariance matrix. This matrix summarizes how sensor noise propagates into estimation errors. By taking the square root of its diagonal values, we find the $\mathbf{1\sigma}$ standard deviation, the range where the error is expected to fall 68% of the time.



The sensor noise floor effectively scales these plots. If we used sensors that were twice as precise, the error magnitude would shrink by half. However, the shape of the error curve, specifically the spike in uncertainty at small separation distances, is dictated by the physical geometry of the robot, not the sensor quality. Better sensors can reduce the scale of the error, but they cannot change the geometric trends.

Crucially, this prediction is theoretical. It represents a "best-case" baseline assuming the noise is purely random. In reality, unmodeled hardware behaviors like drift or plastic deformation of our robot may cause the true experimental error to be higher. These plots show the limits of the geometry, serving as bounds for expected final system performance.